



**UNIVERSIDADE FEDERAL
DE MATO GROSSO**

UNIVERSIDADE FEDERAL DE MATO GROSSO
(UFMT) INSTITUTO DE CIÊNCIAS EXATAS DA
TERRA (ICET)
INTELIGENCIA ARTIFICIAL

JOÃO PAULO ALVES CAMPOS
&
JOÃO PEDRO ZAGONEL

RELATÓRIO DO TRABALHO DE INTELIGÊNCIA
ARTIFICIAL

UNIVERSIDADE FEDERAL DE MATO GROSSO
(UFMT)

JOÃO PAULO ALVES CAMPOS

&

JOÃO PEDRO ZAGONEL

RELATÓRIO DO TRABALHO DE INTELIGÊNCIA
ARTIFICIAL

Trabalho apresentado como requisito à
obtenção de uma das notas parciais da
disciplina de inteligência artificial do curso de
Ciência da Computação da Universidade
Federal de Mato Grosso.

Prof. Dr. Linder Cândido da Silva

Barra do Garças - MT

2026

Sumário

	Páginas
Descrição Geral do Projeto.....	4
1. Problema 1.....	5
2. Arquivos Modificados.....	6
3. Implementação.....	7
Classe ShortestPathProblem.....	7
Uniform Cost Search (UCS).....	7
A-Star (A*).....	7
i. Comparação entre UCS e A*.....	9
ii. Metodologia dos Testes.....	10
iii. Resultados e Análise.....	11
2. Problema 2.....	12
1. implementação:.....	12
i. Construtor.....	12
ii. Função Sucessora.....	12
iii. Classe View.....	12
iv. Novo método em map_util:.....	13
v. Main.....	13
4. Conclusão.....	15
Referências.....	16

Descrição Geral do Projeto

No planejamento de rotas, o objetivo é encontrar a melhor maneira de ir do ponto A ao ponto B (como no Google Maps). Nesta tarefa, você implementará buscas por rotas em Barra do Garças com os algoritmos Uniform Cost Search e A*.

O mapa da cidade de Barra do Garças foi derivado do Open Street Map, e é composto por um conjunto de locais, cada qual possuindo:

- um rótulo único (e.g., 6608996258),
- uma localização geográfica (e.g., latitude=-15.82, longitude=-55.37)
- um conjunto de tags que descreve o tipo de local (e.g., amenity=food).

Existem conexões entre pares de locais. Cada conexão possui uma distância (em metros) e pode ser percorrida em ambas as direções (se a distância de A para B for de 100 metros, então a distância de B para A também será 100 metros).

1. Problema 1

Começamos pelo problema de encontrar o melhor caminho de um ponto de partida (e.g., um ponto na UFMT) até um ponto de chegada (e.g., Águas Quentes). Para este fim, você deve especificar o problema de busca no arquivo `trab-partel.py`, implementar o algoritmo Uniform Cost Search no arquivo `UCS.py` e o algoritmo A* no arquivo `A_start.py`.

2. Arquivos Modificados

Para a resolução do Problema 1 foram realizadas implementações nos seguintes arquivos

Arquivo	Modificação Realizada
trab-part	Implementação da classe ShortestPathProblem , função sucessora e heurística utilizada pelo A*
ucs.py	Implementação completa do algoritmo Uniform Cost Search
a_start.py	Implementação completa do algoritmo A*

3. Implementação

Classe ShortestPathProblem

No arquivo **trab-parte1.py** foi implementada a modelagem do problema de busca por meio da classe **ShortestPathProblem**.

O construtor recebe a localização inicial, a localização de destino e o mapa da cidade, definindo os estados inicial e objetivo da busca.

Também foi implementada a função sucessora, responsável por gerar os estados vizinhos alcançáveis a partir do estado atual, retornando o novo estado, a ação correspondente e o custo da transição.

Além disso, foi implementada a heurística utilizada pelo algoritmo A*, baseada na distância geográfica em linha reta entre o estado atual e o objetivo.

Uniform Cost Search (UCS)

No arquivo **ucs.py** foi implementado o algoritmo Uniform Cost Search utilizando uma fila de prioridade ordenada pelo custo acumulado do caminho.

Foi utilizada a estrutura **reached** para armazenar o melhor caminho conhecido para cada estado visitado, evitando expansões redundantes e garantindo a obtenção da solução ótima.

A-Star (A*)

No arquivo **a_start.py** foi implementado o algoritmo A*, que reutiliza a mesma estrutura do UCS, porém utiliza uma função de prioridade baseada em:

$$f(n) = g(n) + h(n)$$

onde:

- **g(n)** representa o custo acumulado;
- **h(n)** representa a estimativa da distância restante até o objetivo.

Dessa forma, o algoritmo consegue direcionar a busca para regiões mais promissoras do espaço de estados.

i. Comparação entre UCS e A*

As principais diferenças entre os algoritmos estão relacionadas ao uso da heurística. Apesar dessas diferenças, ambos os algoritmos garantem a obtenção de soluções ótimas para o problema.

Características	UCS	A*
Critério de Prioridade	$g(n)$	$g(n) + h(n)$
Usa Heurística	Não	Sim
Direciona Busca	Não	Sim
Estados Explorados	Maior Quantidade	Menor Quantidade
Tempo Execução	Alto	Baixo

ii. Metodologia dos Testes

Foram realizados quinze testes utilizando diferentes pares de origem e destino distribuídos pelo mapa da cidade de Barra do Garças.

Em cada execução foram coletadas as seguintes métricas:

- número de estados explorados;
- tempo de execução;
- custo da solução encontrada.

Os mesmos cenários foram utilizados para ambos os algoritmos, garantindo uma comparação justa dos resultados.

iii. Resultados e Análise

Os resultados mostraram que ambos os algoritmos encontraram caminhos de mesmo custo em todos os testes realizados, confirmando a corretude das implementações.

Entretanto, observou-se uma diferença significativa de desempenho. O UCS explorou em média 5144 estados por execução, enquanto o A* explorou apenas 1107 estados, representando uma redução média de 78,48%.

Em relação ao tempo de execução, o UCS apresentou média de 0,0215 segundos por busca, enquanto o A* apresentou média de 0,0056 segundos.

O maior ganho ocorreu no trajeto entre a Biblioteca da UFMT e o Banco do Brasil, onde o UCS explorou 5192 estados e o A* apenas 347 estados, correspondendo a uma redução superior a 93%.

Esses resultados demonstram que a heurística utilizada pelo A* foi capaz de reduzir significativamente o espaço de busca sem comprometer a qualidade da solução encontrada.

2. Problema 2

Nesta parte você vai implementar uma estratégia que possibilita especificar uma sequência ordenada de pontos pelos quais um trajeto deve passar. Por exemplo, dado uma localização inicial, um conjunto de pontos de passagem e um ponto de destino, o caminho de custo mínimo corresponde ao caminho mais curto do ponto inicial até o ponto final, cobrindo todos os pontos intermediários de passagem. Implemente o problema de busca na classe `WaypointsShortestPathProblem`.

1. implementação:

i. Construtor

Inicialmente o construtor da Classe `WaypointsShortestPathProblem` continua a herdar da Classe `Search problem`, com um novo parâmetro: `waypoints`, uma lista de strings

Waypoints : É uma lista que guardará os paradas que o usuário selecionar dentre os 19 pontos de interesse;

Todos estes pontos de interesse estão mapeados no arquivo *“bg-landmarks.json”*, onde a variável guardará o **valor do par chave-valor** que identifica corretamente a linha correspondente ao ponto de interesse e que posteriormente é feito um casting para o utilização do método da classe `map_util location_from_tag`, para correlacionar a string do ponto de interesse e o tipo **State** usado nos algoritmos de UCS e A*.

ii. Função Sucessora

Não houve necessidade de alterar a função sucessora, portanto, ela foi reutilizada da parte 1 do trabalho. alterar a função sucessora poderia aumentar a complexidade do algoritmo, devido isso, era preferível continuar com a mesma função sucessora e apenas remodelar o problema para que atendesse as necessidades do mesmo.

iii. Classe View

Criada exclusivamente para a parte 2 do trabalho, armazena 2 variáveis: `option_saved` e `waypoints_travelling`.

- **“option_saved”**: É o índice dado pelo usuário que identifica a cidade de onde o usuário

partirá inicialmente.

- **“waipoints_travelling”**: variável que armazena uma lista(String) de waypoints pelos quais o usuário irá viajar.

A classe possui 3 métodos simples que retornam None:

- **Show_menu(self)** : Mostra o menu inicial contendo as opções de operações da main.
- **Show_menu_dot(self)** : Mostra os índices com seus respectivos landmarks/pontos de interesses provindos do “bg_landmarks.json”.
- **Show_route(self)** : Mostra a rota que será realizada , partindo dos dados armazenados na variavel da classe waypoints_travelling.

obs: A classe possui 2 aplicações diferentes no main.

iv. Novo método em map_util:

location_from_tag_id(tag_id): este método foi adicionado para retornar a ponto de interesse correto com base no inteiro que o usuário insere para indicar um ponto de interesse. e.g 6 é o número que faz referencia ao ponto da madre-marta, portanto o com base no número 6 é retornado a string “madre-marta”.

v. Main

A função principal foi implementada com o objetivo de oferecer uma interface interativa ao usuário, permitindo a configuração dinâmica da rota antes de executar o algoritmo de busca.

Após o encerramento do menu, uma das formas pensadas para cobrir todas as paradas/ pontos de interesse foi decompor o problema em uma lista chamada **full_path** que é construída concatenando o ponto inicial com todos os waypoints.

Em seguida, o algoritmo UCS ou A* é executado para cada par consecutivo de pontos realizando o seguinte trecho do código:

```
78  for i in range(len(full_path) - 1):
79      start_state = full_path[i]
80      goal_state = full_path[i + 1]
81      problem = WaypointsShortestPathProblem(start_state, goal_state, waypoints, city_map)
82      ucs.solve(problem)
83      all_actions += ucs.actions
```

Isso garante que o caminho entre cada par de pontos consecutivos seja encontrado individualmente, e as ações de cada trecho são acumuladas em **“all_actions”**.

Ao final da execução, o caminho completo é exibido de duas formas: textualmente via “**print_path**”, que imprime cada localização com suas tags(podem ser um ponto de interesse ou uma tag única gerada pela função **read_map**(osm_path: str)) associadas no terminal , e graficamente via “**plot_map**(city_map: CityMap, path: List[str], waypoint_tags: List[str], map_name: str)”, que renderiza na web o trajeto sobre o mapa recortado de Barra do Garças.

Entre as duas exibições, a rota selecionada pelo usuário é mostrada via **menu.show_route()**, permitindo confirmar visualmente os pontos de passagem antes da abertura do mapa interativo.

4. Conclusão

As implementações realizadas nos arquivos **trab-parte1.py**, **ucs.py** e **a_start.py** permitiram resolver corretamente o problema básico de planejamento de rotas, onde o A* demonstrou superioridade de desempenho devido à heurística baseada em distância geográfica, explorando menos estados e reduzindo o tempo de execução.

Na mesma trajetória, o trabalho evoluiu para o problema de rotas com pontos de passagem (waypoints), abordado no arquivo **trab-parte2.py**, mesmo nesta parte do trabalho a busca A* demonstrou superioridade em questão de desempenho sobre a ucs. A implementação da classe **WaypointsShortestPathProblem** e as adaptações na interface e utilitários permitiram decompor o problema em trechos menores, garantindo uma solução robusta que conecta todos os pontos de interesse selecionados.

Referências

SILVA, Linder Cândido da. **Aulas (1 ... 4):IA_aula.pdf** Material de aula. Barra do Garças: Universidade Federal de Mato Grosso, 2026.

RUSSELL, Stuart; NORVIG, Peter. **Artificial intelligence: a modern approach**. 3. ed. Upper Saddle River: Prentice Hall, 2010.